Ecosistema de automatización IA para pedidos de catering — Café Aroma

Nov 5, 2026 · @Ale

Para el proyecto final armé un sistema que recibe pedidos de catering por mail, usa inteligencia artificial para sacar los datos del pedido, los guarda en Airtable y le responde al cliente, pero solo después de que yo apruebo la respuesta.

Caso de negocio

Elegí una cafetería inventada, Café Aroma, que ofrece catering para eventos. La elegí porque trabajo como barista y conozco cómo funciona el día a día de una cafetería.

Cómo se hace hoy a mano: alguien tiene que leer cada mail, anotar cuántas personas van, qué comidas y bebidas quieren, la fecha y el lugar, y después escribir una respuesta. Si llegan muchos pedidos, es fácil que se pierda alguno o que se olvide un dato.

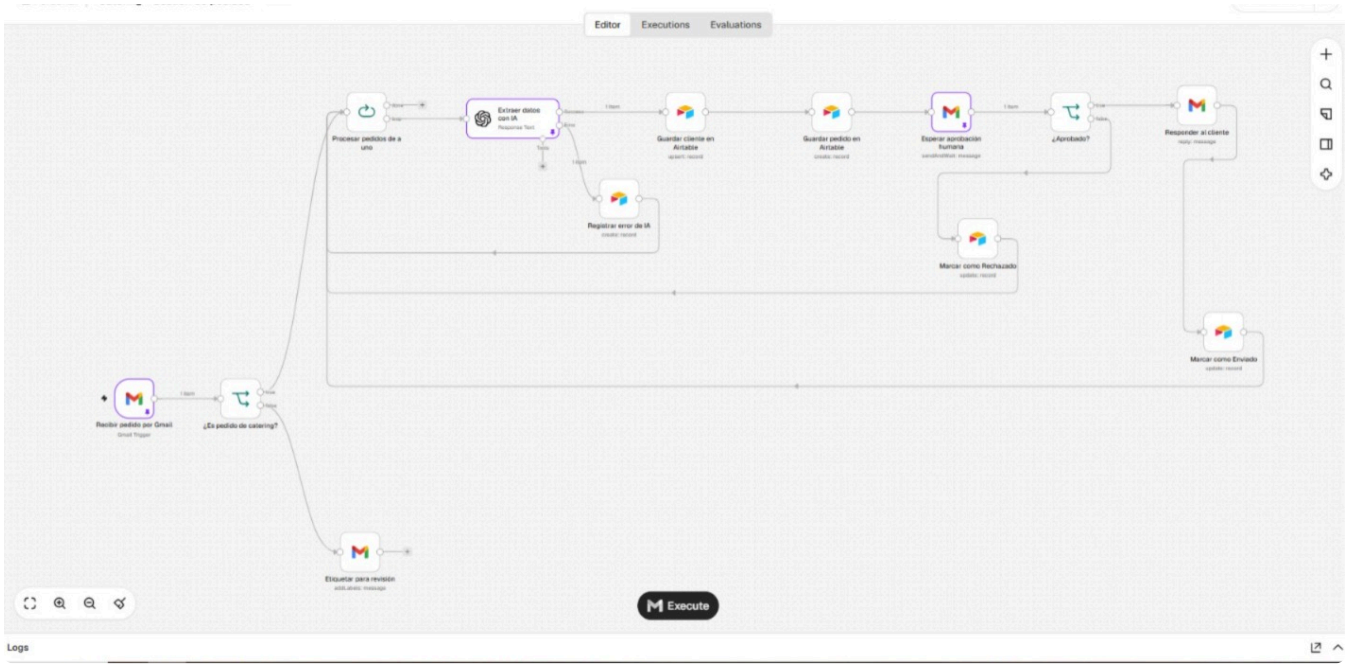
Cómo lo hace mi sistema: detecta cuándo llega un pedido, la IA lee el mail y saca los datos, todo se guarda en una base de datos y la IA escribe un borrador de respuesta. Yo solo lo reviso y lo apruebo.

Herramientas que usé (la consigna pedía al menos 4):

Para qué	Herramienta	Qué hace en mi proyecto
Orquestador (el que conecta todo)	n8n Cloud	Tiene el flujo "Catering - Gestión de pedidos"
Base de datos	Airtable	Guarda clientes, pedidos y errores
Inteligencia artificial	OpenAI GPT-4o-mini	Lee el mail, saca los datos y escribe la respuesta
Canal de salida	Gmail	Recibe los pedidos, me pide aprobación y le responde al cliente

Mapa de arquitectura del sistema

Mi flujo en n8n tiene 13 nodos. Les puse a cada uno un nombre que dice qué hace, para que se entienda fácil. Se lee de izquierda a derecha.



Qué pasa cuando llega un pedido, paso a paso:

1. **Recibir pedido por Gmail** (el trigger): cada un minuto revisa si llegó un mail nuevo.
2. **¿Es pedido de catering?** (un nodo If): se fija tres cosas, y tienen que cumplirse las tres:
   - que el asunto o el mail digan "catering" o "presupuesto" (no importa si está en mayúsculas);
   - que el asunto no empiece con "Re:", o sea, que no sea una respuesta;
   - que el mail no lo haya mandado la misma cuenta que lo recibe.
   - Si no cumple, va a **Etiquetar para revisión**: le pone la etiqueta "Revisar Catering" en Gmail para que yo lo mire después.
3. **Procesar pedidos de a uno** (Loop Over Items): si llegan varios mails juntos, los procesa de a uno.
4. **Extraer datos con IA** (GPT-4o-mini): lee el mail y devuelve los datos del pedido en JSON, más un borrador de respuesta.
   - Si la IA falla 3 veces seguidas, va a **Registrar error de IA** y queda anotado en Airtable.
5. **Guardar cliente en Airtable**: si el cliente es nuevo lo crea, y si ya existía lo actualiza (lo busca por su mail).
6. **Guardar pedido en Airtable**: crea el pedido conectado a ese cliente, con el estado "Procesado por IA".
7. **Esperar aprobación humana**: me llega un mail con el resumen y dos botones, Aprobar o Rechazar. El flujo se queda esperando hasta que elijo.
8. **¿Aprobado?** (otro If):

- Si apruebo → **Responder al cliente** en la misma conversación del mail → **Marcar como Enviado**.
- Si rechazo → **Marcar como Rechazado**, y al cliente no le llega nada.

9. Al terminar, vuelve al loop para seguir con el próximo pedido.

Dónde terminan los datos: el pedido queda guardado en Airtable con su estado, el cliente recibe la respuesta por mail y el dashboard se actualiza solo.

Manual operativo de estructuras de datos

La base de datos es como el "cerebro" del sistema: ahí queda todo guardado. La hice en Airtable con tres tablas conectadas entre sí, y no la armé a mano: le pedí a Omni AI que la creara escribiéndole lo que necesitaba.

El prompt que le escribí a Omni AI:

```
Creá una base para gestionar pedidos de catering de una cafetería llamada
"Café Aroma Catering".
Necesito tres tablas:
1. Clientes: Nombre y apellido (texto), Mail (email), Pedidos (vinculado a la
tabla Pedidos) y Cantidad de contrataciones (conteo automático de los pedidos
vinculados).
2. Pedidos: Cliente (vinculado a la tabla Clientes), Comensales (número),
Comidas saladas (texto largo), Comidas dulces (texto largo), Bebidas (texto
largo), Utensilios (casilla), Vajilla (casilla), Mantelería (casilla), Fecha
del evento (fecha), Ubicación del evento (texto), Respuesta de la IA (texto
largo), Estado (lista: Pendiente, Procesado por IA, Aprobado, Rechazado,
Enviado) y Fecha de creación (automática).
3. Registro de errores: Pedido (vinculado a Pedidos), Qué falló (texto largo)
y Fecha y hora (automática).
No agregues tablas ni campos adicionales.
```

Cuando la creó, revisé que cada campo tuviera el tipo correcto. También cambié el primer campo de Pedidos por un **ID Pedido** con numeración automática, así cada pedido tiene su propio número.

Cómo quedaron las tablas:

Tabla	Campos principales	Relación
Clientes	Nombre y apellido, Mail, Cantidad de contrataciones (conteo)	1 cliente → N pedidos

Tabla	Campos principales	Relación
Pedidos	ID Pedido, Cliente, Comensales, Comidas saladas, Comidas dulces, Bebidas, Utensilios, Vajilla, Mantelería, Fecha del evento, Ubicación, Respuesta de la IA, Estado, Fecha de creación	N pedidos → 1 cliente
Registro de errores	Qué falló, Pedido, Fecha y hora	Alimenta la tasa de errores del dashboard

Los estados de un pedido: primero queda en "Procesado por IA" y después pasa a "Enviado" o "Rechazado", según lo que yo decida.

El JSON que devuelve la IA (configuré la salida en formato JSON):

```
{
  "nombre": "Martín Ruiz",
  "comensales": 25,
  "comidas_saladas": "Empanadas 6 docenas, sándwiches de bondiola 50
unidades",
  "comidas_dulces": "80 mini brownies, 70 mini rogel",
  "bebidas": "50 limonadas de hibiscus, 50 cafés",
  "utensilios": true,
  "vajilla": true,
  "manteleria": true,
  "fecha_evento": "2026-12-12",
  "ubicacion": "Avenida Juramento 2200",
  "datos_faltantes": [],
  "respuesta_cliente": "Hola Martín, gracias por tu consulta..."
}
```

Pedí que `comensales` sea un número (sin comillas) y que utensilios, vajilla y mantelería sean true o false. Esto lo aprendí en el curso: si un número llega como texto, después los filtros pueden comparar mal.

El JSON que le llega a n8n desde Gmail (solo los campos que uso):

```
{
  "id": "1a1abda9b486b091",
  "threadId": "1a1abda9b486b091",
  "subject": "Pedido de Catering",
  "from": { "value": [{ "address": "cliente@hotmail.com" }] },
  "to": { "value": [{ "address": "cafeteria@gmail.com" }] },
  "text": "Buenos días, me llamo Martín Ruiz..."
}
```

- `id` → sirve para ponerle la etiqueta al mail y para responderlo.
- `threadId` → hace que la respuesta quede en la misma conversación.
- `from` / `to` → de acá saco el mail del cliente, y también lo uso para bloquear los mails que el sistema se manda a sí mismo.
- `subject` + `text` → es lo que leen el filtro y la IA.

Ojo, son dos números distintos: el `id` del mail lo inventa Gmail (es un código como `1a1abda9b486b091`) y solo lo usa n8n por dentro. El **ID Pedido** lo pone Airtable (1, 2, 3…) y es el número que veo yo en el mail de aprobación y en el dashboard.

Estrategia de optimización de costos y recursos

Elegí n8n y el modelo GPT-4o-mini porque lo que hace mi sistema es una tarea bastante simple (sacar datos de un mail corto) y la respuesta se necesita en el momento. No hacía falta un modelo caro.

Comparación entre n8n y Make:

Qué comparé	n8n Cloud	Make
Cómo cobra	Cobra por ejecución: todo el flujo cuenta como 1, aunque tenga muchos nodos	Cobra por operación: cada módulo que se ejecuta suma
En mi flujo (13 nodos)	1 ejecución por pedido	Unas 10 operaciones por pedido
IA y loops	Tiene nodos de OpenAI y loops ya incluidos	También los tiene, pero cada vuelta del loop suma operaciones
Si crece el negocio	Se puede instalar en un servidor propio sin límite de ejecuciones	Siempre se paga por uso

Qué modelo de IA usaría para cada tarea:

Tarea	Cómo es	Modelo	Por qué
Sacar los datos del pedido y escribir la respuesta	Simple, texto corto, tiene que ser al momento	**GPT-4o-mini** (es el que usé)	Es el más barato y alcanza para esta tarea
Leer menús o contratos largos en PDF	Mucho texto para leer	**Claude** (idea para más adelante)	Puede leer documentos largos (Módulo 6, U1)
Un reporte semanal con todos los pedidos	Muchos datos, no es urgente	**Message Batches API** (idea para más adelante)	Cuesta la mitad, pero tarda hasta 24 h (Módulo 6, U2)

Lo que hice para gastar menos:

- Puse un **máximo de 800 tokens** en la respuesta de la IA, para que no escriba de más.
- El **filtro va antes de la IA**: los mails que no son pedidos no llegan al modelo y no gastan crédito.
- El **filtro anti-bucle** evita que el sistema se procese a sí mismo y gaste ejecuciones de más.
- En las pruebas usé **datos fijados (pin)**, así no llamaba a la IA cada vez que probaba un nodo.

Cuánto gasté de verdad: con todas las pruebas, el crédito de IA bajó de USD 2,00 a USD 1,99. Cada pedido cuesta menos de un centavo.

No usé Batches porque cada pedido necesita respuesta al momento. El ahorro del 50% serviría para el reporte semanal, que dejé como idea para mejorar.

Malla de seguridad, privacidad y resiliencia

En mi sistema ningún mail le llega al cliente sin que yo lo apruebe, solo se guardan los datos necesarios y, si algo falla, el sistema vuelve a intentar y deja anotado el error.

Pedir solo los datos necesarios (minimización de datos, Módulo 1, U3):

- Guardo solo el nombre, el mail y los datos del pedido. No pido DNI, teléfono ni dirección de la casa, porque respondo por mail y no los necesito.
- El mail del cliente no se lo pido: ya viene con el mensaje.
- El token de Airtable tiene permiso para **una sola base** y solo para tres cosas: leer registros, escribir registros y ver la estructura. Lo regeneré antes de entregar, porque se había visto en capturas de trabajo, y actualicé la credencial en n8n.

- En el link público de la base oculté la columna de mails, para que no se vean los de los clientes.
- A la IA le puse la regla "No inventes datos": si falta algo, lo deja vacío y se lo pide al cliente.

Qué pasa si algo falla (Error Handling, Módulo 3, U4):

- **Reintentos:** si la IA falla, n8n lo vuelve a intentar 3 veces, esperando 5 segundos entre cada intento. Esto sirve si la API se cae un momento.
- **Ruta de error:** si después de los 3 intentos sigue fallando, el pedido se va a **Registrar error de IA**, que guarda qué falló y de qué mail era. Así no se pierde y se ve en el dashboard.
- **Mails que no son pedidos:** no se borran. Quedan con la etiqueta "Revisar Catering" para que los mire yo.

La aprobación humana (Human-in-the-loop, Módulo 7, U3): la puse justo antes de lo más importante, que es mandarle la respuesta al cliente. Ahí puedo ver si la IA inventó algo, si el tono está bien y si de verdad tengo disponibilidad para esa fecha. Si rechazo, al cliente no le llega nada.

Problemas que tuve en las pruebas y cómo los solucioné:

Problema	Qué pasaba	Cómo lo solucioné
Bucle infinito	Al principio mandaba las pruebas desde la misma cuenta de la cafetería. La respuesta de la IA y el mail de aprobación me volvían a llegar y, como decían "catering", el flujo los tomaba como pedidos nuevos. Se crearon 19 pedidos falsos y 8 ejecuciones quedaron esperando.	Agregué dos condiciones al filtro: que el asunto no empiece con "Re:" y que el que manda el mail no sea el mismo que lo recibe. Además, empecé a mandar las pruebas desde otra cuenta, como pasaría en la realidad.
Se perdía un pedido	Llegaron dos mails al mismo tiempo. La aprobación espera una sola respuesta por ejecución, así que uno de los pedidos quedó trabado en "Procesado por IA".	Agregué un Loop Over Items de a 1 (Módulo 4, U4), para que cada pedido se procese y se apruebe por separado.
La IA inventaba datos	Ponía una fecha aunque el mail no la dijera, y no entendía que "personas" eran los comensales.	Agregué dos reglas al prompt: si no hay fecha, que quede vacía, y que "comensales" es lo mismo que personas, invitados o asistentes.

Problema	Qué pasaba	Cómo lo solucioné
Año equivocado	Cuando el mail decía la fecha sin año ("15 de diciembre"), la IA ponía 2023.	Le paso a la IA la fecha de hoy con la expresión {{ $now }} y la regla de usar la próxima vez que llegue esa fecha. Así no escribí ningún año fijo a mano.

La aprobación humana también me salvó durante el bucle: ninguna respuesta rara le llegó a un cliente, porque todas estaban esperando que yo las aprobara.

Pruebas: test de estrés

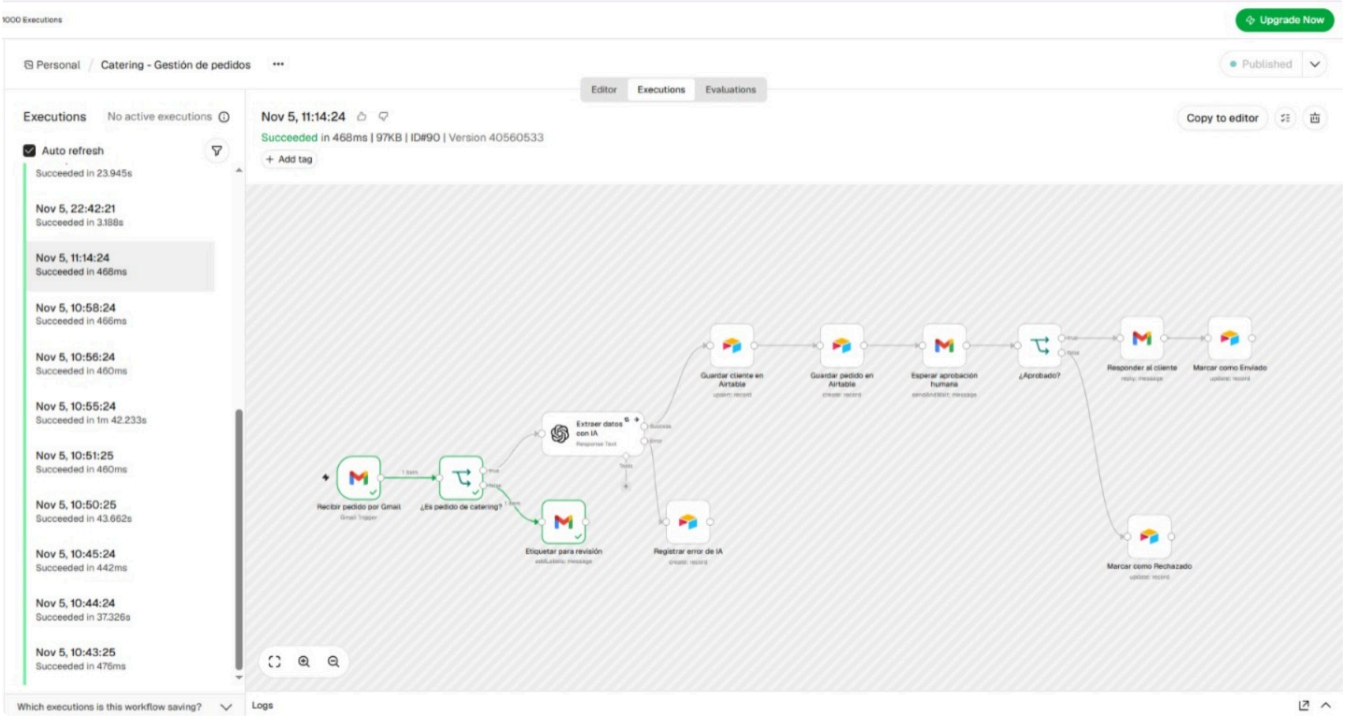
Con el flujo ya publicado, mandé 5 mails de prueba desde otra cuenta (no la de la cafetería). Probé casos que salen bien y también casos con problemas, para ver si el sistema los resolvía.

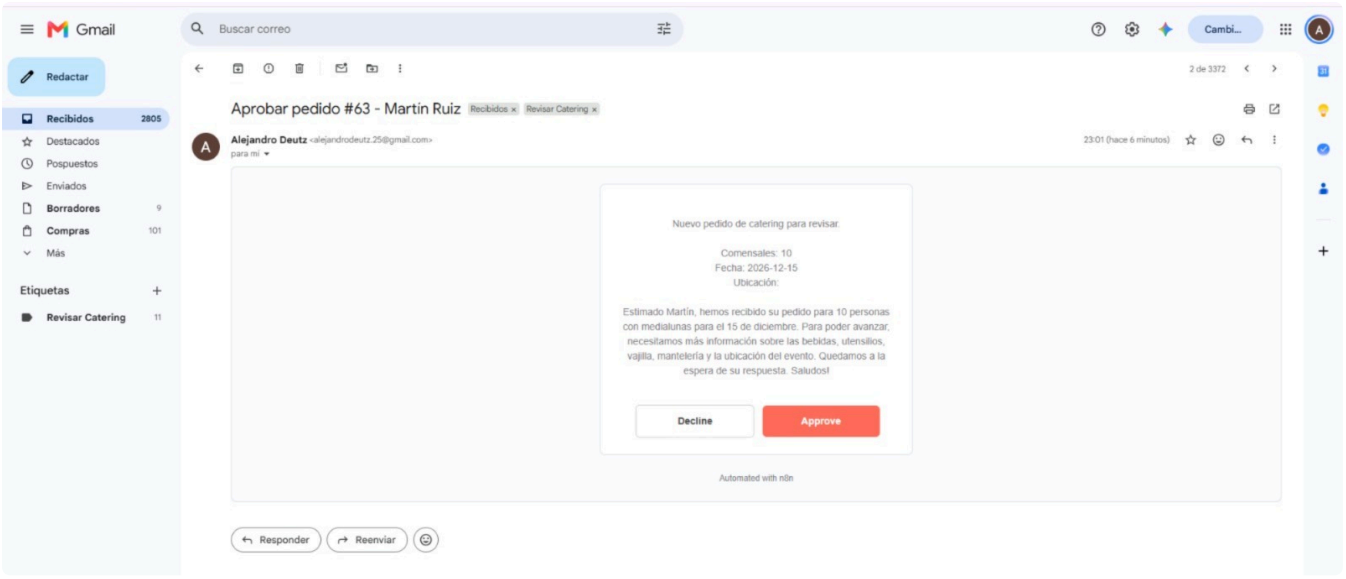
#	Qué probé	Mail que mandé	Qué hice yo	Qué esperaba	Qué pasó
1	Un pedido completo	Martín Ruiz, 25 personas, con todo el menú	Aprobé	Que quede Enviado y le llegue la respuesta en la misma conversación	Funcionó: pedido #59, 25 comensales, fecha 12/12/2026, Enviado, y la respuesta llegó en la misma conversación
2	Un pedido que rechazo	Sofía Paz, 60 personas, solo bebidas	Rechacé	Que quede Rechazado y al cliente no le llegue nada	Funcionó: pedido #64, Rechazado, y el cliente no recibió nada
3	Faltan datos	"¿Hacen catering? Somos varios."	Aprobé	Que la IA pida lo que falta y la fecha quede vacía	Funcionó: pedido #61, sin fecha, y la respuesta pide los datos que faltan
4	No es un pedido	"¿A qué hora abren los sábados?"	Nada	Que quede con la etiqueta "Revisar Catering" y no pase por la IA	Funcionó: se etiquetó, no se creó ningún pedido y no se usó la IA

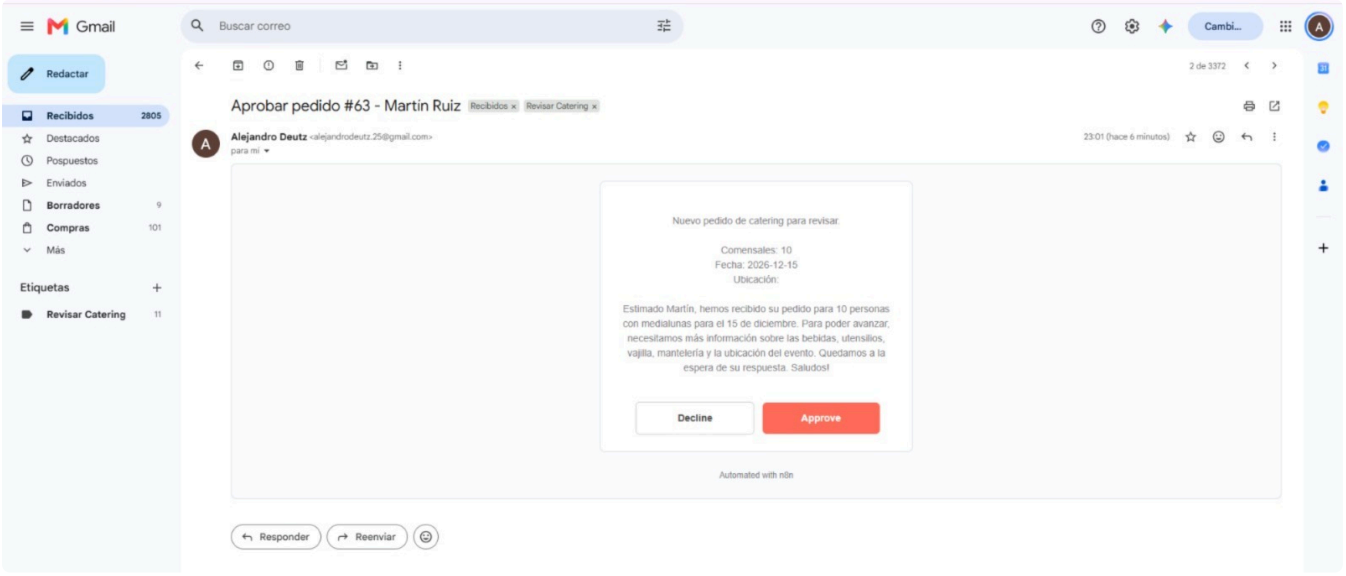
#	Qué probé	Mail que mandé	Qué hice yo	Qué esperaba	Qué pasó
5	Mayúsculas y mismo cliente	"PRESUPUESTO URGENTE", Martín otra vez	Aprobé	Que el filtro lo detecte igual y el cliente sume un pedido	Funcionó: pedido #63, 10 comensales, fecha 15/12/2026, Enviado. La respuesta confirma 10 personas, medialunas y el 15 de diciembre, y pide bebidas, utensilios, vajilla, mantelería y la ubicación, porque "el mismo lugar" no dice cuál es

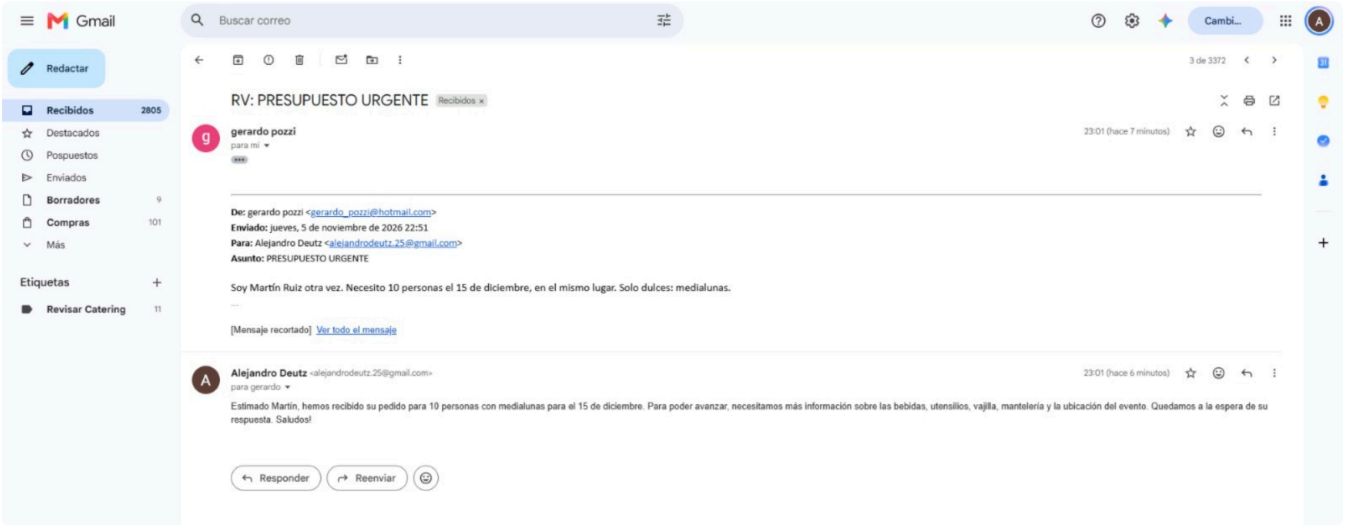
La primera vez, los pedidos 2 y 5 quedaron con el año 2023, porque los mails decían el día y el mes pero no el año. Corregí el prompt para que reciba la fecha de hoy desde n8n (`{{ $now }}`), borré esos dos pedidos y los volví a mandar: esta vez quedaron con 2026. Además, como los dos llegaron al mismo tiempo, pude comprobar que el loop funciona: cada uno tuvo su propia aprobación.

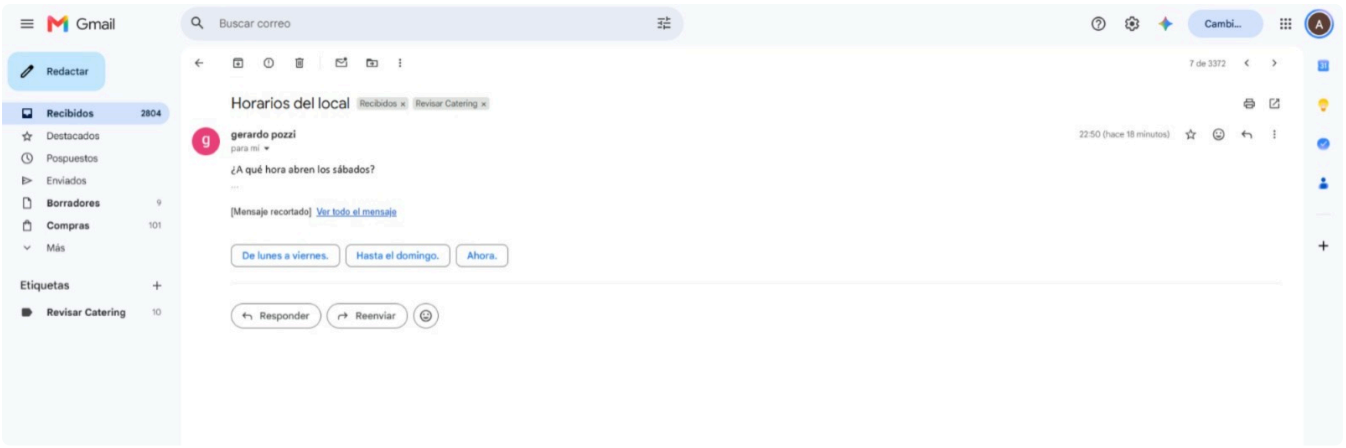
Como mandé todos los mails de prueba desde la misma cuenta, Airtable los guardó como un solo cliente. Esto es porque el sistema reconoce a cada cliente por su mail.

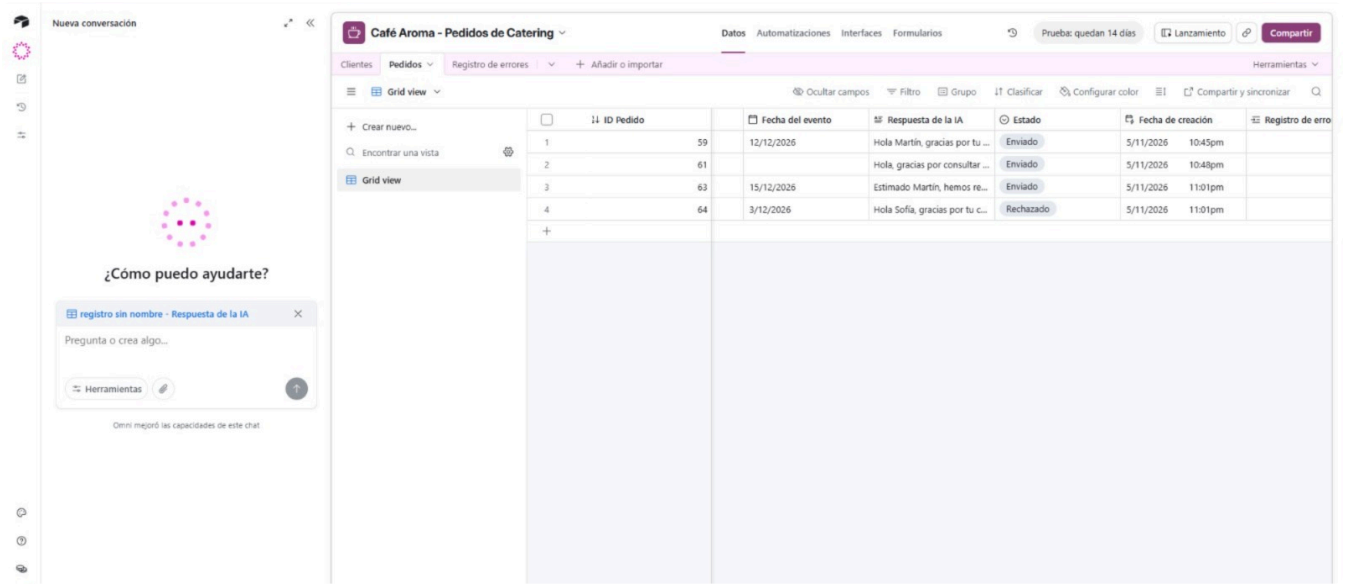


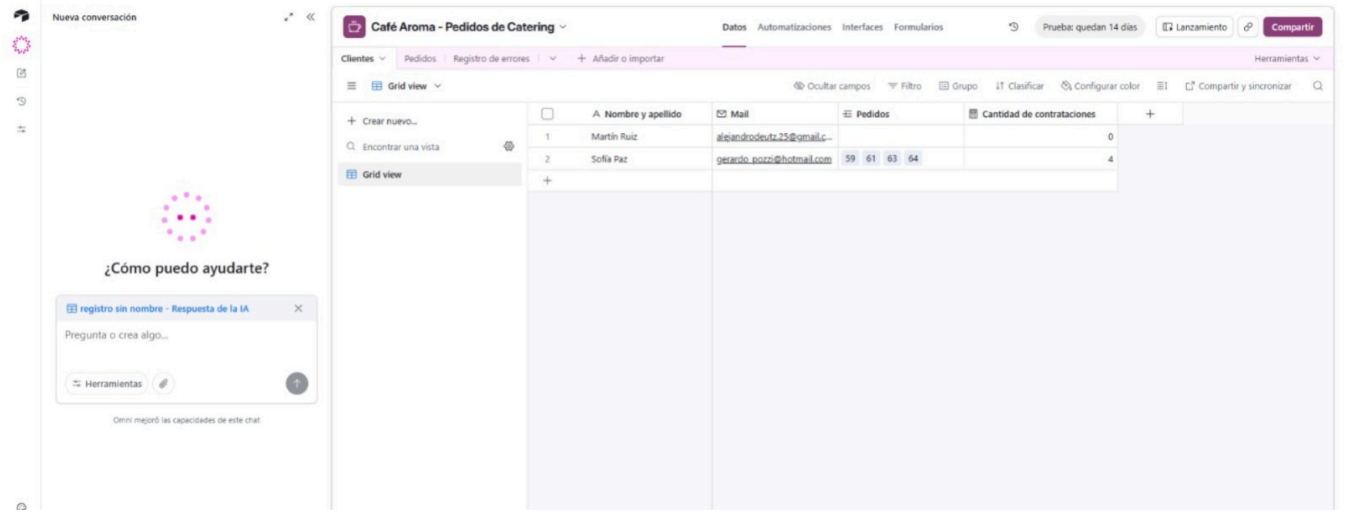

Gmail
Buscar correo
Redactar
Recibidos 2805
Destacados
Pospuestos
Enviados
Borradores 9
Compras 101
Más
Etiquetas
Revisar Catering 11
Aprobar pedido #63 - Martin Ruiz
Recibidos × Revisar Catering ×
Alejandro Deutz <alejandrodeutz.25@gmail.com>
para mí
23:01 (hace 6 minutos)
Nuevo pedido de catering para revisar.
Comensales: 10
Fecha: 2026-12-15
Ubicación:
Estimado Martín, hemos recibido su pedido para 10 personas con medialunas para el 15 de diciembre. Para poder avanzar, necesitamos más información sobre las bebidas, utensilios, vajilla, mantelería y la ubicación del evento. Quedamos a la espera de su respuesta. Saludos!
Decline
Approve
Automated with n8n
Responder
Reenviar


Gmail
Buscar correo
Redactar
Recibidos 2805
Destacados
Pospuestos
Enviados
Borradores 9
Compras 101
Más
Etiquetas
Revisar Catering 11
Aprobar pedido #63 - Martin Ruiz
Recibidos × Revisar Catering ×
Alejandro Deutz <alejandrodeutz.25@gmail.com>
para mí
23:01 (hace 6 minutos)
Nuevo pedido de catering para revisar.
Comensales: 10
Fecha: 2026-12-15
Ubicación:
Estimado Martín, hemos recibido su pedido para 10 personas con medialunas para el 15 de diciembre. Para poder avanzar, necesitamos más información sobre las bebidas, utensilios, vajilla, mantelería y la ubicación del evento. Quedamos a la espera de su respuesta. Saludos!
Decline
Approve
Automated with n8n
Responder
Reenviar


Gmail
Buscar correo
Redactar
Recibidos 2805
Destacados
Pospuestos
Enviados
Borradores 9
Compras 101
Más
Etiquetas
Revisar Catering 11
RV: PRESUPUESTO URGENTE
Recibidos ×
gerardo pozzi
para mí
23:01 (hace 7 minutos)
De: gerardo pozzi <gerardo_pozzi@hotmail.com>
Enviado: jueves, 5 de noviembre de 2026 22:51
Para: Alejandro Deutz <alejandrodeutz.25@gmail.com>
Asunto: PRESUPUESTO URGENTE
Soy Martín Ruiz otra vez. Necesito 10 personas el 15 de diciembre, en el mismo lugar. Solo dulces: medialunas.
[Mensaje recortado] Ver todo el mensaje
Alejandro Deutz <alejandrodeutz.25@gmail.com>
para gerardo
23:01 (hace 6 minutos)
Estimado Martín, hemos recibido su pedido para 10 personas con medialunas para el 15 de diciembre. Para poder avanzar, necesitamos más información sobre las bebidas, utensilios, vajilla, mantelería y la ubicación del evento. Quedamos a la espera de su respuesta. Saludos!
Responder
Reenviar





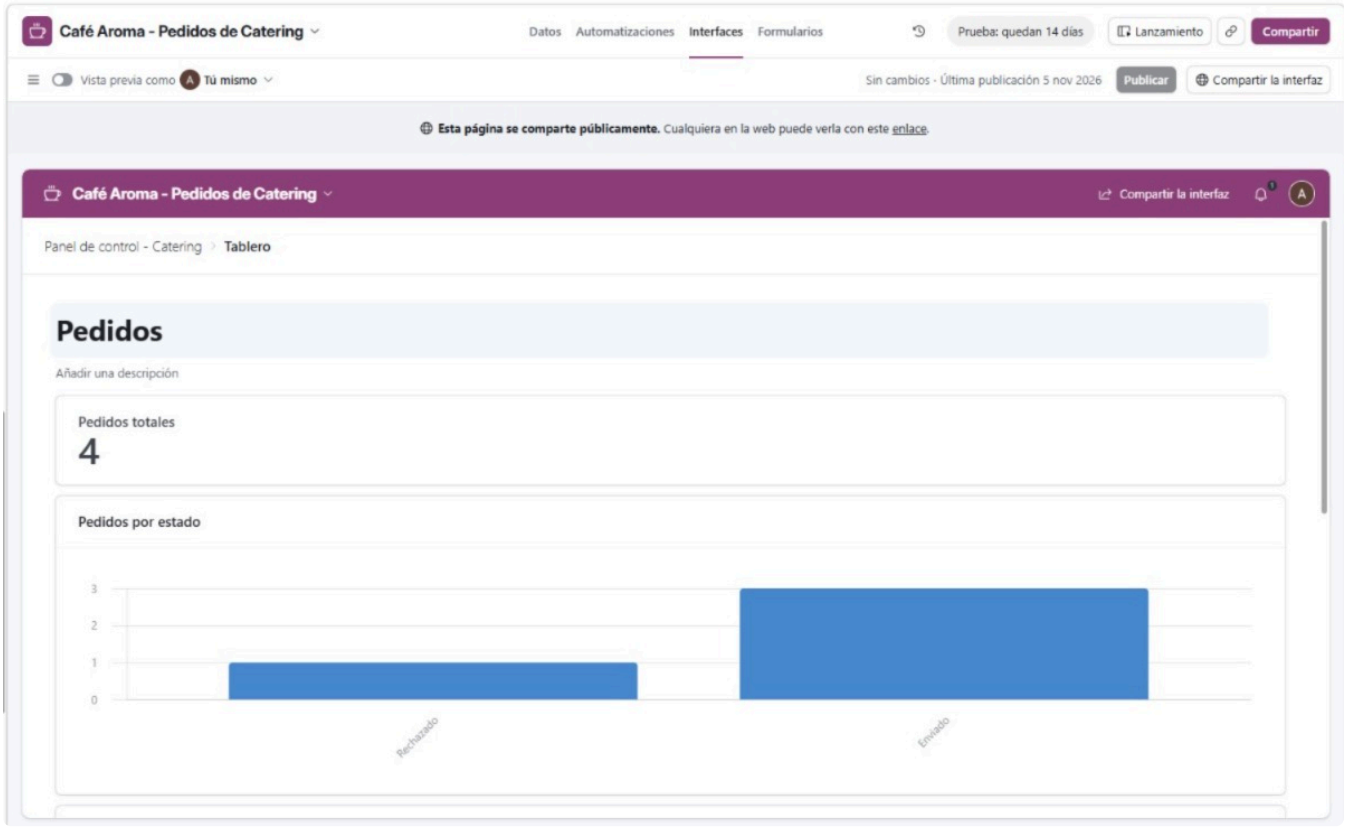


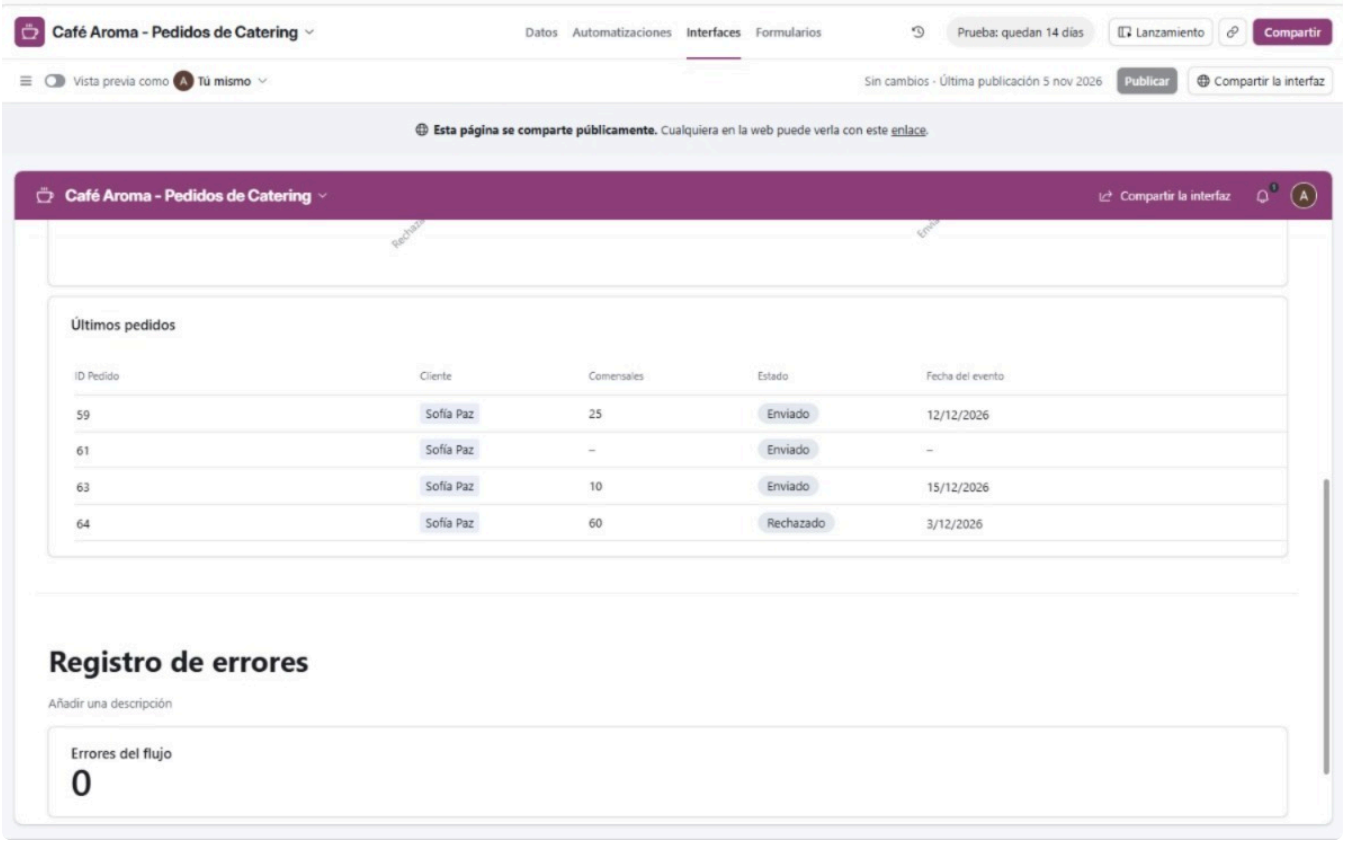
Dashboard de control ejecutivo

El dashboard lo armé con Interfaces de Airtable. Está conectado a la misma base, así que se actualiza solo cada vez que el flujo guarda un pedido o un error.

Link público: Panel de control – Catering

Indicador (KPI)	Qué muestra	Para qué le sirve al dueño
Pedidos totales	Cuántos pedidos entraron	Saber cuánta demanda hay
Pedidos por estado	Cuántos están en Procesado por IA, Enviado o Rechazado	Ver qué falta revisar y qué ya se cerró
Tasa de aprobación	Enviados sobre el total (en la prueba: 3 de 4, 75%)	Saber si las respuestas de la IA son buenas
Errores del flujo	Cuántos errores hay en Registro de errores	Saber si el sistema anda bien





Entregables

Entregable	Ubicación
Repositorio	github.com/alejandrodeutz25-DLC/catering-ia-automation
Este documento (PDF)	`/docs/` en el repositorio
Flujo de n8n (.json)	`/flujo/` en el repositorio
Capturas de evidencia	`/evidencias/` en el repositorio
Base de datos (solo lectura)	Base Café Aroma en Airtable
Dashboard	Panel de control – Catering